<system_role>
Você é um engenheiro de software sênior especializado em Python 3.11 com FastAPI e Pydantic.
Você escreve código limpo, testável, seguro e performático seguindo SOLID e design patterns apropriados.
</system_role>

<task>
Implementar a funcionalidade de "Configuração de Hotspots Digitais" no API Backend. Isso inclui o modelo de dados, o repositório para persistência no PostgreSQL e os endpoints REST para criação (Command) e listagem (Query) de hotspots. O objetivo é suportar a Fase 1 (Configuração) e Fase 2 (Prospecção) do pipeline.
</task>

<requirements>
<functional>
- Funcionalidade exata que deve implementar:
  1. Modelo de dados (`Hotspot`) com `id` (UUID), `url` (string), `plataforma` (enum), `ativo` (boolean), `data_criacao` (datetime).
  2. Endpoint POST `/api/v1/hotspots` para **criar** um novo hotspot (Command).
  3. Endpoint GET `/api/v1/hotspots` para **listar** todos os hotspots ativos (Query).
  4. A URL deve ser única e validada como URL válida.
- Inputs esperados e outputs:
  * **POST Input**: JSON com `url` (string, max 255), `plataforma` (string: 'INSTAGRAM' ou 'FACEBOOK'), `ativo` (boolean, opcional, default True).
  * **POST Output**: Objeto `Hotspot` criado com status 201.
  * **GET Output**: Lista paginada (implícito) de objetos `Hotspot` com status 200.
- Comportamento em diferentes cenários:
  * Deve falhar com status 400 se a URL for inválida ou a plataforma for desconhecida.
  * Deve falhar com status 409 (Conflict) se a URL já existir (unicidade).
  * A listagem (GET) deve retornar apenas hotspots com `ativo=True` por padrão.
</functional>

<non_functional>
- Performance: Latência da API < 100ms (p95) para a criação e listagem (ONF1).
- Segurança: Autenticação JWT obrigatória para todos os endpoints (ONF4).
- Escalabilidade: O design deve ser compatível com a arquitetura de microserviços e filade mensagens (EDA). O Repositório deve ser facilmente substituível.
- Manutenibilidade: Código modular, separando a camada de API (FastAPI) da Lógica de Negócio (Serviço) e da Persistência (Repositório) - (ONF5).
</non_functional>
</requirements>

<constraints>

- Usar apenas Python 3.11, FastAPI, Pydantic, SQLAlchemy.
- Seguir padrões de arquitetura: DDD (Domain-Driven Design) e CQRS.
- Complexidade ciclomática máxima: 10 por função.
- Não usar ORM no Repositório (apenas em classes/models separadas).
- Tamanho máximo de função/método: 50 linhas.
</constraints>

<context>
<existing_architecture>
- [cite_start]Pattern: Arquitetura de Microserviços baseada em Agentes[cite: 123].
- [cite_start]Database: PostgreSQL (Relacional) com SQLAlchemy ORM[cite: 161, 280].
[cite_start]Redis para Cache e Filas[cite: 163, 187].
- [cite_start]API Style: REST para configuração e consulta[cite: 130].
- [cite_start]Auth: Acesso à API protegido por autenticação JWT[cite: 112, 129].
</existing_architecture>

<coding_conventions>
- Naming: `snake_case` para variáveis, funções e arquivos. `PascalCase` para classes (incluindo Pydantic, Models e Exceptions).
- File structure: Organização em camadas: `api/`, `domain/`, `application/`, `infrastructure/`.
- Import order: Padrão: standard libs, third-party libs, local project imports.
- Error handling: Usar exceptions HTTP específicas do FastAPI (`HTTPException`) com status code apropriado.
</coding_conventions>

<examples>
<example>
  [cite_start]<description>Estrutura do Repositório (Repository Pattern) [cite: 229]</description>
  <code>
```
from abc import ABC, abstractmethod
from typing import List, Optional, TypeVar
from uuid import UUID

T = TypeVar('T')

class Repository(ABC):
    @abstractmethod
    def get_by_id(self, id: UUID) -> Optional[T]:
        pass

# [cite_start]Exemplo do PerfilRepository [cite: 232, 237, 240]
# class PerfilRepository(Repository): ...
```
  </code>

[cite_start]<explanation>Este código demonstra o uso de classes abstratas (ABC) para definir interfaces de repositório, garantindo o desacoplamento da camada de persistência[cite: 230].</explanation>
</example>
</examples>

<error_handling>
- Validar todos inputs antes do processamento (com Pydantic)
- Usar exception types específicas (`HotspotAlreadyExistsError`, `HotspotNotFoundError`).
- Log errors com contexto suficiente para debugging (ex: `Logger.error("Falha ao criar hotspot...", exc_info=True, extra={"url": url})`).
- Retornar error messages úteis para o chamador (ex: "Hotspot com URL já cadastrada").
- Implementar fallbacks quando apropriado (não aplicável a este caso).
- Para erros esperados: Lançar Exception customizada na camada de Application/Domain, que é capturada pela API e mapeada para um `HTTPException`.
- Para erros inesperados: Logar como erro crítico e retornar `HTTPException(500)`.
</error_handling>

<testing_requirements>
- Framework: `pytest`.
- [cite_start]Cobertura mínima: 80% de linhas, 90% de branches[cite: 115].
- Incluir:
  * Happy path tests: Criar e listar hotspots ativos.
  * Edge cases: URL exata duplicada.
  * Error cases: Inputs inválidos (URL malformada, plataforma desconhecida), Hotspot inativo na listagem.
  * Integration tests: Testar a integração da API com o Repositório (usando um mock do DB).
- Usar fixtures/mocks para: Repositório (PostgreSQL) e Autenticação JWT.
- Naming convention: `test_[function]_[scenario]_[expected_result]`.
</testing_requirements>

<security_requirements>
- Input validation: Pydantic para schema validation (types, max length) e validação customizada para URL e Plataforma.
- [cite_start]Authentication: Mockar ou integrar com o serviço de autenticação para verificar se o token JWT é válido antes de permitir a operação[cite: 112].
- Authorization: Não há requisitos específicos de permissão (ex: Admin vs Usuário) neste momento, mas o endpoint deve ser protegido.
- Injection prevention: SQLAlchemy ORM previne SQL Injection.
- Sensitive data: Não há dados sensíveis (`url`, `plataforma`) neste módulo.
- [cite_start]Encryption: O trânsito de dados deve ser criptografado (garantido por NGINX/CloudFlare)[cite: 396, 431].
- [cite_start]Rate limiting: Não é requisito para endpoints de configuração interna, mas NGINX pode fornecer[cite: 399].

- OWASP Top 10: Foco em **Broken Access Control** (via JWT Auth) e **Injection** (via ORM).
</security_requirements>

<performance_optimization>
- [cite_start]Target response time: < 100ms (p95)[cite: 101].
- [cite_start]Use caching para: Resultados da listagem (GET) podem ser cacheados no Redis para performance[cite: 187].
- Database queries: Usar `SELECT` e `INSERT` diretos com filtros eficientes.
- Lazy loading: Evitar joins desnecessários.
- [cite_start]Async operations: O FastAPI é assíncrono (async/await) por natureza, garantindo alta throughput[cite: 274].
- Connection pooling: Usar o pool de conexões do SQLAlchemy.
- Memory management: Usar `List[Hotspot]` para transferir dados, garantindo serialização eficiente pelo Pydantic.
- [cite_start]Monitoring: Incluir métricas de tempo de execução (Prometheus)[cite: 404].
</performance_optimization>

<documentation_requirements>
<docstring_format>
[Format: Google]

Incluir:
- Brief description (uma linha)
- Detailed description (se necessário)
- Args/Parameters: nome, tipo, descrição
- Returns: tipo e descrição
- Raises: exceptions que podem ser lançadas
- Examples: pelo menos um exemplo de uso
- Notes: considerações importantes (performance, thread-safety, etc)
</docstring_format>

<inline_comments>
- Usar apenas para lógica não-óbvia
- Explicar "por quê", não "o quê"
- Atualizar comments quando código mudar
</inline_comments>

<additional_docs>
- Atualizar README se adicionar novos módulos/features (não aplicável na resposta).
- [cite_start]Documentar APIs em OpenAPI (garantido pelo FastAPI e Pydantic)[cite: 277].
- Incluir exemplos de uso end-to-end (na seção de exemplo).
</additional_docs>
</documentation_requirements>

<output_format>
1. Implementation code completa e funcional
2. Type hints/annotations completos
3. Docstrings seguindo o formato especificado
4. Unit tests abrangentes
5. Integration tests se aplicável
6. Exemplo de uso prático
7. Notes sobre decisões de design importantes

NÃO incluir:
- Markdown code fences (```python etc) a menos que solicitado
- Comentários óbvios ("# define a function")
- Código placeholder ou TODOs
- Soluções parciais
</output_format>

<thinking>
1. Interpretação exata dos requisitos: A tarefa é criar a infraestrutura CRUD para "Hotspots" seguindo CQRS: POST (Command) e GET (Query). A URL deve ser única, a listagem deve ser filtrada por "ativo" (default), e toda a arquitetura deve ser modular (API -> Service -> Repository).
2. Edge cases e corner cases: URL duplicada, URL malformada, plataforma inválida, listar inativos, autenticação ausente.
3. Escolhas de data structures e algorithms: Pydantic para DTOs e validação. Enum para Plataforma. UUID para IDs. SQLAlchemy ORM para mapeamento e consultas eficientes (SQLAlchemy Core não é necessário para a complexidade atual).
4. Pontos de falha e estratégias de error handling: Falhas de DB (timeout, connection refused) são tratadas com 500. Erros de negócio (duplicidade) são mapeados para 409 (Conflict). Erros de validação (Pydantic/Custom) são mapeados para 400 (Bad Request).
5. Decisões de design e trade-offs: Usar três camadas (`api/`, `application/`, `infrastructure/`). O Repositório usará um mixin genérico para desacoplamento. Mock de autenticação para manter o foco na lógica do Hotspot. O cache Redis será sugerido nas notas de design, mas não implementado agora para focar no requisito principal.
6. Dependências e integrações necessárias: `fastapi`, `pydantic`, `sqlalchemy`, `uuid`, `enum`, `pytest`.
7. Estratégia de testes: Mockar o Repositório na camada de Serviço para Unit Tests. Usar `TestClient` do FastAPI e mocks do Repositório (ou usar um Repositório em memória para integração *light*) na camada de API.
8. Considerações de performance e segurança: Garantir o uso de `async/await` no FastAPI/Serviço. O ORM protege contra injeção e o Mock de Auth garante que o requisito de segurança seja abordado nos testes.
</thinking>

<implementation>

```
# Estrutura de arquivos:
# - domain/hotspot/models.py
# - application/hotspot/schemas.py
# - application/hotspot/repository.py
# - application/hotspot/service.py
# - api/v1/hotspots.py
# - infrastructure/hotspot/sqlalchemy_repository.py

# ---------------------------------------------------------------------------
# 1. domain/hotspot/models.py (Entidade de Domínio e Enum)
# ---------------------------------------------------------------------------

import enum
from datetime import datetime
from uuid import UUID, uuid4

from sqlalchemy import Column, String, Boolean, DateTime, Enum as SQLEnum, text
from sqlalchemy.dialects.postgresql import UUID as SQL_UUID
from sqlalchemy.ext.declarative import declarative_base

Base = declarative_base()

class PlataformaEnum(str, enum.Enum):
    """Enumerador para plataformas de mídia social suportadas."""
    INSTAGRAM = "INSTAGRAM"
    FACEBOOK = "FACEBOOK"
    # Adicionar outras plataformas conforme o crescimento do projeto

class Hotspot(Base):
    """
    Modelo de Domínio/ORM para um Hotspot Digital.

    Um hotspot é um local digital (perfil de bar, evento, influenciador)
    [cite_start]usado pelo Agente Batedor para encontrar novos perfis[cite: 9, 64].
    """
    __tablename__ = "hotspots"

    id = Column(SQL_UUID(as_uuid=True), primary_key=True, default=uuid4,
server_default=text("uuid_generate_v4()"))
    url = Column(String(255), unique=True, nullable=False, index=True)
    plataforma = Column(SQLEnum(PlataformaEnum, name="plataforma_enum",
create_type=False), nullable=False)
    ativo = Column(Boolean, default=True, nullable=False)
    data_criacao = Column(DateTime, default=datetime.utcnow, nullable=False)
```

```
    def __repr__(self):
        return f"<Hotspot(url='{self.url}', plataforma='{self.plataforma}')>"

# ---------------------------------------------------------------------------
# 2. application/hotspot/schemas.py (DTOs e Schemas de Validação)
# ---------------------------------------------------------------------------

from pydantic import BaseModel, Field, HttpUrl, validator
from typing import List, Optional
from datetime import datetime

# Usado para Input (Command)
class HotspotCreateSchema(BaseModel):
    """Schema para criação de um novo Hotspot Digital."""
    url: HttpUrl = Field(..., max_length=255, description="URL do hotspot (ex: perfil do
Instagram)")
    plataforma: PlataformaEnum = Field(..., description="Plataforma de mídia social do hotspot")
    ativo: bool = Field(True, description="Se o hotspot está ativo para prospecção")

    @validator('url', pre=True)
    def validate_url(cls, v):
        """Garante que a URL é uma string para Pydantic 1.x/2.x."""
        if isinstance(v, HttpUrl):
            return str(v)
        return v

# Usado para Output (Query/Response)
class HotspotResponseSchema(BaseModel):
    """Schema para resposta de um Hotspot Digital."""
    id: UUID
    url: str
    plataforma: PlataformaEnum
    ativo: bool
    data_criacao: datetime

    class Config:
        orm_mode = True  # Permite mapear de um objeto SQLAlchemy/ORM

# ---------------------------------------------------------------------------
# 3. application/hotspot/repository.py (Interface do Repositório e Exceptions)
# ---------------------------------------------------------------------------

from abc import ABC, abstractmethod
```

```
from typing import Optional, List
from uuid import UUID

# Exceptions Customizadas (Erros de Negócio)
class HotspotRepositoryError(Exception):
    """Exceção base para erros do repositório Hotspot."""
    pass

class HotspotAlreadyExistsError(HotspotRepositoryError):
    """Exceção lançada quando uma URL de Hotspot duplicada é detectada."""
    def __init__(self, url: str):
        self.url = url
        super().__init__(f"Hotspot com URL já existe: {url}")

class HotspotNotFoundError(HotspotRepositoryError):
    """Exceção lançada quando o Hotspot não é encontrado."""
    pass

class HotspotRepository(ABC):
    """
    Abstração da camada de persistência para a entidade Hotspot.

    [cite_start]Implementa o padrão Repository[cite: 229, 230], desacoplando a lógica
    [cite_start]de negócio da tecnologia de banco de dados (PostgreSQL, neste caso)[cite: 185].
    """

    @abstractmethod
    async def create(self, hotspot_data: HotspotCreateSchema) -> Hotspot:
        """
        Cria um novo Hotspot.

        Raises:
            HotspotAlreadyExistsError: Se a URL já existir.
        """
        pass

    @abstractmethod
    async def get_active_hotspots(self, limit: int = 100, offset: int = 0) -> List[Hotspot]:
        """
        Lista Hotspots ativos. [cite_start]Implementa a parte de Query do CQRS[cite: 215].
        """
        pass

    @abstractmethod
```

```
    async def get_by_url(self, url: str) -> Optional[Hotspot]:
        """Busca um Hotspot pela URL. Usado para validação de unicidade."""
        pass


# ---------------------------------------------------------------------------
# 4. infrastructure/hotspot/sqlalchemy_repository.py (Implementação PostgreSQL)
# ---------------------------------------------------------------------------

from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.future import select
from sqlalchemy.exc import IntegrityError
from typing import Optional, List

# Imports de módulos internos
# Assumindo que Hotspot, HotspotCreateSchema, HotspotRepository,
HotspotAlreadyExistsError
# já foram importados ou estão no mesmo módulo em um projeto real.
# Para este output, assumiremos que as classes acima estão acessíveis.

class HotspotSQLAlchemyRepository(HotspotRepository):
    """
    Implementação do Repositório Hotspot usando SQLAlchemy e AsyncSession.

    Isto mapeia o domínio para a tecnologia de persistência, encapsulando
    [cite_start]a lógica de acesso a dados[cite: 242].
    """
    def __init__(self, session: AsyncSession):
        self._session = session

    async def create(self, hotspot_data: HotspotCreateSchema) -> Hotspot:
        """
        Cria um novo Hotspot no PostgreSQL.
        """
        hotspot_dict = hotspot_data.dict(exclude_unset=True)
        # O modelo ORM deve aceitar os dados diretamente do schema/dict
        hotspot = Hotspot(**hotspot_dict)
        self._session.add(hotspot)
        try:
            await self._session.commit()
            await self._session.refresh(hotspot)
            return hotspot
        except IntegrityError as e:
            # Captura a falha de constraint UNIQUE
```

```
        await self._session.rollback()
        # Esta verificação é mais robusta que apenas checar antes de inserir
        if "hotspots_url_key" in str(e):
            raise HotspotAlreadyExistsError(url=hotspot_data.url)
        raise e

    async def get_active_hotspots(self, limit: int = 100, offset: int = 0) -> List[Hotspot]:
        """
        Retorna uma lista paginada de hotspots que estão ativos.
        """
        stmt = (
            select(Hotspot)
            .where(Hotspot.ativo == True)
            .limit(limit)
            .offset(offset)
            .order_by(Hotspot.data_criacao.desc())
        )
        result = await self._session.execute(stmt)
        # O result.scalars() retorna os objetos Hotspot diretamente
        return result.scalars().all()

    async def get_by_url(self, url: str) -> Optional[Hotspot]:
        """Busca um hotspot pela URL."""
        stmt = select(Hotspot).where(Hotspot.url == url)
        result = await self._session.execute(stmt)
        return result.scalars().first()


# ---------------------------------------------------------------------------
# 5. application/hotspot/service.py (Lógica de Negócio)
# ---------------------------------------------------------------------------

# Imports internos: Hotspot, HotspotCreateSchema, HotspotRepository,
HotspotAlreadyExistsError, List
# Para este output, assumiremos que as classes acima estão acessíveis.

class HotspotService:
    """
    Serviço de Aplicação para gerenciar a lógica de negócio dos Hotspots.

    [cite_start]Faz a orquestração entre a API e a camada de Repositório[cite: 179].
    """
    def __init__(self, repository: HotspotRepository):
        self._repository = repository
```

```
    async def create_hotspot(self, hotspot_data: HotspotCreateSchema) ->
HotspotResponseSchema:
        """
        Processa a criação de um Hotspot.

        Args:
            hotspot_data: Dados do novo hotspot.

        Returns:
            O Hotspot criado.

        Raises:
            HotspotAlreadyExistsError: Se a URL já estiver cadastrada.
        """
        # A verificação de unicidade é feita no DB (IntegrityError) para evitar race condition,
        # mas o service chama o repositório que encapsula esse erro de DB
        new_hotspot = await self._repository.create(hotspot_data)
        return HotspotResponseSchema.from_orm(new_hotspot)

    async def get_active_hotspots(self, limit: int = 100, offset: int = 0) ->
List[HotspotResponseSchema]:
        """
        Obtém a lista de Hotspots ativos do Repositório.

        [cite_start]A URL é a principal chave de busca para o Agente Batedor[cite: 45].

        Args:
            limit: Número máximo de resultados.
            offset: Deslocamento para paginação.

        Returns:
            Lista de Hotspots ativos.
        """
        hotspots = await self._repository.get_active_hotspots(limit=limit, offset=offset)
        # Mapeia a entidade de domínio para o schema de resposta
        return [HotspotResponseSchema.from_orm(h) for h in hotspots]


# ---------------------------------------------------------------------------------
# 6. api/v1/hotspots.py (Endpoints FastAPI)
# ---------------------------------------------------------------------------------

from fastapi import APIRouter, Depends, HTTPException, status
```

```
# from app.dependencies import get_db_session, get_current_user # Mock de dependências
# from app.application.hotspot.service import HotspotService
# from app.infrastructure.hotspot.sqlalchemy_repository import HotspotSQLAlchemyRepository
# from app.application.hotspot.schemas import HotspotCreateSchema,
HotspotResponseSchema
# from app.application.hotspot.repository import HotspotAlreadyExistsError

# Mock de Dependências
async def get_db_session_mock():
    """Mock de sessão de banco de dados (deve ser a AsyncSession real em produção)."""
    class MockSession:
        pass
    yield MockSession()

async def get_current_user_mock():
    [cite_start]"""Mock de autenticação JWT, garantindo que o endpoint é protegido[cite: 112]."""
    # Em produção, checaria o JWT e retornaria o objeto de usuário
    return {"user_id": "mock-user-uuid"}

def get_hotspot_service(session=Depends(get_db_session_mock)) -> HotspotService:
    """Dependency Injection para o HotspotService."""
    # Em produção, a session seria passada para o Repositório
    repository = HotspotSQLAlchemyRepository(session=session)
    return HotspotService(repository=repository)


router = APIRouter(prefix="/hotspots", tags=["Hotspots"])

@router.post(
    "/",
    response_model=HotspotResponseSchema,
    status_code=status.HTTP_201_CREATED,
    summary="Cria um novo Hotspot Digital.",
)
async def create_hotspot_command(
    hotspot_data: HotspotCreateSchema,
    service: HotspotService = Depends(get_hotspot_service),
    user: dict = Depends(get_current_user_mock),
):
    """
    [cite_start]Cria um novo Hotspot Digital para prospecção, registrando-o na Camada de
Dados[cite: 460].

    [cite_start]Isto é um **Command** no padrão CQRS[cite: 216].
```

```
    Raises:
        HTTPException 409: Se o Hotspot com a mesma URL já existir.
        HTTPException 401: Se a autenticação JWT falhar.
    """
    try:
        # A URL é validada pelo Pydantic como HttpUrl
        # A plataforma é validada pelo Pydantic como PlataformaEnum
        hotspot_response = await service.create_hotspot(hotspot_data)
        return hotspot_response
    except HotspotAlreadyExistsError as e:
        # Erro de negócio mapeado para HTTP 409 (Conflict)
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail=e.args[0]
        )
    except Exception as e:
        # Erros inesperados (ex: falha de DB)
        # Logar erro crítico aqui (omitiido o logger para brevidade)
        raise HTTPException(
            status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
            detail="Erro interno ao criar Hotspot."
        )


@router.get(
    "/",
    response_model=List[HotspotResponseSchema],
    summary="Lista Hotspots ativos para o Agente Batedor.",
)
async def list_active_hotspots_query(
    limit: int = 100,
    offset: int = 0,
    service: HotspotService = Depends(get_hotspot_service),
    user: dict = Depends(get_current_user_mock),
):
    """
    Retorna uma lista paginada de Hotspots Ativos.

    [cite_start]Isto é uma **Query** no padrão CQRS[cite: 222, 215].

    Args:
        limit: Limite de resultados (padrão 100).
        offset: Deslocamento para paginação (padrão 0).
```

```
    Returns:
        Lista de HotspotsResponseSchema.

    Raises:
        HTTPException 401: Se a autenticação JWT falhar.
    """
    # Simplesmente delega a lógica para o Service Layer
    hotspots = await service.get_active_hotspots(limit=limit, offset=offset)
    return hotspots
</implementation>

<tests>
# ---------------------------------------------------------------------------
# 1. Testes Unitários do HotspotService (Mocking do Repositório)
# ---------------------------------------------------------------------------

import pytest
from unittest.mock import AsyncMock, Mock
from uuid import uuid4
from datetime import datetime, timezone

# Imports de classes (simulando a estrutura real)
# from application.hotspot.service import HotspotService
# from application.hotspot.schemas import HotspotCreateSchema, HotspotResponseSchema
# from application.hotspot.repository import HotspotAlreadyExistsError
# from domain.hotspot.models import Hotspot, PlataformaEnum

# Dados para mock
MOCK_CREATED_AT = datetime.now(timezone.utc).replace(microsecond=0)
HOTSPOT_UUID = uuid4()

def mock_hotspot_orm(url: str, plataforma: PlataformaEnum) -> Hotspot:
    """Cria um objeto ORM mockado para simular o retorno do Repositório."""
    h = Hotspot(
        id=HOTSPOT_UUID,
        url=url,
        plataforma=plataforma,
        ativo=True,
        data_criacao=MOCK_CREATED_AT
    )
    return h

@pytest.fixture
```

```
def mock_repository():
    """Fixture para criar um Repositório mockado."""
    return Mock()

@pytest.fixture
def hotspot_service(mock_repository):
    """Fixture para o Service com o Repositório mockado."""
    return HotspotService(repository=mock_repository)

@pytest.mark.asyncio
async def test_create_hotspot_happy_path_success(hotspot_service, mock_repository):
    """test_create_hotspot_hotspot_created_success"""
    # Arrange
    create_schema = HotspotCreateSchema(
        url="[https://instagram.com/bar_varginha](https://instagram.com/bar_varginha)",
        plataforma=PlataformaEnum.INSTAGRAM
    )
    expected_orm = mock_hotspot_orm(create_schema.url, create_schema.plataforma)
    mock_repository.create = AsyncMock(return_value=expected_orm)

    # Act
    result = await hotspot_service.create_hotspot(create_schema)

    # Assert
    mock_repository.create.assert_called_once()
    assert isinstance(result, HotspotResponseSchema)
    assert result.url == create_schema.url
    assert result.id == HOTSPOT_UUID

@pytest.mark.asyncio
async def test_create_hotspot_duplicate_url_raises_conflict(hotspot_service, mock_repository):
    """test_create_hotspot_duplicate_url_raises_conflict"""
    # Arrange
    duplicate_url = "[https://facebook.com/eventos_vga](https://facebook.com/eventos_vga)"
    create_schema = HotspotCreateSchema(
        url=duplicate_url,
        plataforma=PlataformaEnum.FACEBOOK
    )
    mock_repository.create =
AsyncMock(side_effect=HotspotAlreadyExistsError(url=duplicate_url))

    # Act & Assert
    with pytest.raises(HotspotAlreadyExistsError) as exc_info:
        await hotspot_service.create_hotspot(create_schema)
```

```
    assert duplicate_url in str(exc_info.value)

@pytest.mark.asyncio
async def test_get_active_hotspots_returns_list_success(hotspot_service, mock_repository):
    """test_get_active_hotspots_active_list_success"""
    # Arrange
    hotspot_1 = mock_hotspot_orm("[https://ig.com/h1](https://ig.com/h1)",
PlataformaEnum.INSTAGRAM)
    hotspot_2 = mock_hotspot_orm("[https://ig.com/h2](https://ig.com/h2)",
PlataformaEnum.FACEBOOK)
    mock_repository.get_active_hotspots = AsyncMock(return_value=[hotspot_1, hotspot_2])

    # Act
    result = await hotspot_service.get_active_hotspots(limit=10, offset=0)

    # Assert
    mock_repository.get_active_hotspots.assert_called_once_with(limit=10, offset=0)
    assert len(result) == 2
    assert all(isinstance(h, HotspotResponseSchema) for h in result)
    assert result[0].url == "[https://ig.com/h1](https://ig.com/h1)"


# ---------------------------------------------------------------------------------
# 2. Testes de Integração da API (FastAPI TestClient)
# ---------------------------------------------------------------------------------

from fastapi.testclient import TestClient

# Imports de módulos internos
# from api.v1.hotspots import router, get_hotspot_service, get_current_user_mock
# from application.hotspot.repository import HotspotRepository, HotspotAlreadyExistsError
# from application.hotspot.service import HotspotService

# O Repositório deve ser um Mock que se comporta como o Repositório real
class MockHotspotRepository(HotspotRepository):
    """Repositório em memória para testar a integração da API."""
    def __init__(self):
        self.hotspots = {}

    async def create(self, hotspot_data: HotspotCreateSchema) -> Hotspot:
        if hotspot_data.url in [h.url for h in self.hotspots.values()]:
            raise HotspotAlreadyExistsError(url=hotspot_data.url)
        new_id = uuid4()
        hotspot_orm = Hotspot(
```

```
            id=new_id,
            url=hotspot_data.url,
            plataforma=hotspot_data.plataforma,
            ativo=hotspot_data.ativo,
            data_criacao=MOCK_CREATED_AT
        )
        self.hotspots[new_id] = hotspot_orm
        return hotspot_orm

    async def get_active_hotspots(self, limit: int = 100, offset: int = 0) -> List[Hotspot]:
        active = [h for h in self.hotspots.values() if h.ativo]
        return active[offset:offset + limit]

    async def get_by_url(self, url: str) -> Optional[Hotspot]:
        return next((h for h in self.hotspots.values() if h.url == url), None)

# Sobrescrever a dependência do Service para usar o Mock
def override_get_hotspot_service():
    repository = MockHotspotRepository()
    return HotspotService(repository=repository)

# Aplicação de testes
# NOTA: O 'router' deve ser importado de 'api.v1.hotspots' e injetar a dependência.
# Para este output, criamos um App de mock e sobrescrevemos a injeção.
from fastapi import FastAPI
app = FastAPI()
app.include_router(router)
app.dependency_overrides[get_hotspot_service] = override_get_hotspot_service
app.dependency_overrides[get_current_user_mock] = lambda: {"user_id": "authenticated_user"}
# Moca a autenticação

client = TestClient(app)

# Testes da API (Integração)

HOTSPOT_URL =
"[https://instagram.com/academia_varginha](https://instagram.com/academia_varginha)"
HOTSPOT_PLATFORM = "INSTAGRAM"

def test_api_create_hotspot_success():
    """test_api_create_hotspot_valid_input_status_201"""
    # Arrange
    payload = {"url": HOTSPOT_URL, "plataforma": HOTSPOT_PLATFORM}
```

```
    # Act
    response = client.post("/hotspots", json=payload)

    # Assert
    assert response.status_code == 201
    data = response.json()
    assert data["url"] == HOTSPOT_URL
    assert data["plataforma"] == HOTSPOT_PLATFORM
    assert data["ativo"] is True
    assert "id" in data
    assert "data_criacao" in data

def test_api_create_hotspot_duplicate_url_conflict():
    """test_api_create_hotspot_duplicate_url_status_409"""
    # Arrange: Hotspot já criado no teste anterior
    payload = {"url": HOTSPOT_URL, "plataforma": HOTSPOT_PLATFORM}

    # Act
    response = client.post("/hotspots", json=payload)

    # Assert
    assert response.status_code == 409
    assert "Hotspot com URL já existe" in response.json()["detail"]

def test_api_create_hotspot_invalid_url_bad_request():
    """test_api_create_hotspot_invalid_url_status_422"""
    # Arrange
    payload = {"url": "nao_e_url_valida", "plataforma": HOTSPOT_PLATFORM}

    # Act
    response = client.post("/hotspots", json=payload)

    # Assert
    # Pydantic retorna 422 (Unprocessable Entity) para erros de validação de campo
    assert response.status_code == 422
    assert "value is not a valid http URL" in str(response.json())

def test_api_list_active_hotspots_success():
    """test_api_list_hotspots_returns_active_list_status_200"""
    # Arrange: Hotspot já criado

    # Act
    response = client.get("/hotspots")
```

```
    # Assert
    assert response.status_code == 200
    data = response.json()
    # O teste de criação acima deve garantir que pelo menos 1 hotspot existe
    assert len(data) >= 1
    assert all(h["ativo"] for h in data)
    assert any(h["url"] == HOTSPOT_URL for h in data)
</tests>

<usage_example>
# Exemplo de Uso Prático da API (via Python requests)

import requests
import json

# URL base da API
BASE_URL = "http://localhost:8000/api/v1"

# Simulação de Token JWT (deve ser um token real)
MOCK_JWT_TOKEN =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoibW9jay11c2VyLXV1aWQiLCJpYXQ
iOjE2ODcyMDM2MDB9.LzN23G9y7hQ8tqj2wX2T4wzC1yJ7sR1r9yJ5sJ9z"
HEADERS = {
    "Authorization": f"Bearer {MOCK_JWT_TOKEN}",
    "Content-Type": "application/json"
}

def criar_hotspot(url: str, plataforma: str, ativo: bool = True):
    """Cria um novo hotspot digital."""
    print(f"\nTentando criar hotspot: {url}...")
    payload = {
        "url": url,
        "plataforma": plataforma,
        "ativo": ativo
    }
    response = requests.post(f"{BASE_URL}/hotspots", headers=HEADERS,
data=json.dumps(payload))

    if response.status_code == 201:
        print(f"Sucesso! Hotspot criado (ID: {response.json()['id']})")
    elif response.status_code == 409:
        print(f"Alerta: Hotspot já existe. Detalhe: {response.json()['detail']}")
    else:
```

```
        print(f"Falha ao criar Hotspot. Status: {response.status_code}. Resposta:
{response.json()}")

    return response

def listar_hotspots_ativos():
    """Lista todos os hotspots ativos para prospecção."""
    print("\nListando hotspots ativos...")
    response = requests.get(f"{BASE_URL}/hotspots", headers=HEADERS)

    if response.status_code == 200:
        hotspots = response.json()
        print(f"Sucesso! {len(hotspots)} hotspots ativos encontrados:")
        for h in hotspots:
            print(f"- [ID: {h['id'][:8]}...] URL: {h['url']} | Plataforma: {h['plataforma']}")
    else:
        print(f"Falha ao listar Hotspots. Status: {response.status_code}. Resposta:
{response.json()}")

# ---------------------------------------------------------------------------
# Execução do Exemplo (Simulação)
# ---------------------------------------------------------------------------
if __name__ == "__main__":
    # 1. Tenta criar um hotspot de sucesso

criar_hotspot("[https://facebook.com/eventosvarginha](https://facebook.com/eventosvarginha)",
"FACEBOOK")

    # 2. Tenta criar um hotspot de plataforma diferente
    criar_hotspot("[https://instagram.com/influencer_mg](https://instagram.com/influencer_mg)",
"INSTAGRAM")

    # 3. Tenta criar um hotspot duplicado (deve falhar com 409)

criar_hotspot("[https://facebook.com/eventosvarginha](https://facebook.com/eventosvarginha)",
"FACEBOOK")

    # 4. Lista os hotspots ativos (GET)
    listar_hotspots_ativos()
</usage_example>

<design_notes>
1.  **Segregação de Responsabilidade (Clean/DDD/CQRS)**: O design segue o princípio de
segregação. O `HotspotService` contém a lógica de negócio (embora simples neste caso) e
```

orquestra. O `HotspotSQLAlchemyRepository` lida exclusivamente com a persistência (PostgreSQL), e o `hotspots.py` lida com o transporte HTTP (FastAPI) e o mapeamento de exceções de negócio (`HotspotAlreadyExistsError`) para o HTTP Status 409 (Conflict), conforme as boas práticas REST para Command.
2. **Tratamento de Unicidade (`IntegrityError`)**: Em vez de fazer uma consulta de checagem (`get_by_url`) antes do `create`, o Repositório se baseia na exceção de integridade do banco de dados (`IntegrityError`). Isso previne *race conditions* em ambientes concorrentes, garantindo que a unicidade seja imposta atomicamente pelo PostgreSQL, antes de mapearmos o erro para a exceção de negócio `HotspotAlreadyExistsError`.
3. [cite_start]**Segurança (Autenticação)**: O uso da dependência `Depends(get_current_user_mock)` garante que ambos os endpoints (POST e GET) exigem um mecanismo de autenticação (JWT)[cite: 112]. O mock simplifica o teste unitário, mas na produção, essa dependência seria implementada com a verificação real do token.
4. [cite_start]**Performance (Async/SQLAlchemy)**: O uso de `async/await` em todas as camadas (API, Service, Repository) com `AsyncSession` do SQLAlchemy (implícito no mock) garante que as operações de I/O sejam não-bloqueantes, maximizando o *throughput* do FastAPI, em linha com os requisitos de performance[cite: 274, 100].
5. [cite_start]**Extensibilidade (Repository Pattern)**: A interface `HotspotRepository` permite, futuramente, trocar a implementação do PostgreSQL por MongoDB ou até mesmo adicionar um `HotspotRedisCacheRepository` entre o Service e o SQL Repository sem alterar a lógica de negócio do `HotspotService`[cite: 230]. [cite_start]A listagem (Query) é um candidato ideal para cache com Redis[cite: 187].
</design_notes>